CodInClub

Powered By BridgeLabz

Practice Problems for Exception Handling

1. Checked Exception (Compile-time Exception)

💡 **Problem Statement:**

Create a Java program that reads a file named `"data.txt"`. If the file does not exist, handle the `IOException` properly and display a user-friendly message.

Expected Behavior:

- If the file exists, print its contents.
- If the file does not exist, catch the `IOException` and print `"File not found"`.

```
package com.checkedexception;

import java.io.*;

public class CheckedException {
   public static void main(String[] args) {
       // Defining the file to be read
       File file = new File("src/main/java/com/checkedexception/data.txt");

       try {
           // Creating a FileReader and BufferedReader to read the file
           FileReader fr = new FileReader(file);
           BufferedReader br = new BufferedReader(fr);

           // Reading and printing the file contents line by line
           String line;
           System.out.println("File contents:");
           while ((line = br.readLine()) != null) {
               System.out.println(line);
           }

           // Closing the BufferedReader
```

```
            br.close();

        } catch (IOException e) {
            // Handling IOException if the file does not exist or cannot be read
            System.out.println("File not found. Please check the file name and try
again.");
        }
    }
}
```

```
package com.checkedexception;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.io.*;

public class CheckedExceptionTest {

    @Test
    void testFileExists() {
        // Checking if the file exists
        File file = new File("src/main/java/com/checkedexception/data.txt");
        assertTrue(file.exists(), "File should exist");
    }

    @Test
    void testFileReading() {
        // Reading the file and checking its content
        File file = new File("src/main/java/com/checkedexception/data.txt");
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line = br.readLine();
            assertNotNull(line, "File should contain at least one line");
        } catch (IOException e) {
            fail("IOException should not be thrown");
        }
    }

    @Test
    void testFileNotFound() {
        // Testing behavior when the file does not exist
        File file = new File("src/main/java/com/checkedexception/nonexistent.txt");
        IOException exception = assertThrows(IOException.class, () -> {
```

```
            try (BufferedReader br = new BufferedReader(new FileReader(file))) {
                br.readLine();
            }
        });
        assertNotNull(exception.getMessage(), "IOException should be thrown with a
message");
    }
}
```

---

2. Unchecked Exception (Runtime Exception)

💡 **Problem Statement:**

Write a Java program that asks the user to enter two numbers and divides them. Handle possible exceptions such as:

- **`ArithmeticException`** if division by zero occurs.
- **`InputMismatchException`** if the user enters a non-numeric value.

Expected Behavior:

- If the user enters valid numbers, print the result of the division.
- If the user enters `0` as the denominator, catch and handle `ArithmeticException`.
- If the user enters a non-numeric value, catch and handle `InputMismatchException`.

```
package com.uncheckedexception;

import java.util.Scanner;

public class UncheckedException {
```

```
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        try {
            // Asking the user to enter two numbers
            System.out.println("Enter the first number: ");
            int num1 = input.nextInt();

            System.out.println("Enter the second number: ");
            int num2 = input.nextInt();

            // Performing division and printing the result
            int result = num1 / num2;
            System.out.println("Result: " + result);

        } catch (ArithmeticException e) {
            // Handling division by zero
            System.out.println("Error: Division by zero is not allowed.");
        } catch (Exception e) {
            // Handling non-numeric input
            System.out.println("Error: Please enter a valid numeric value.");
        } finally {
            input.close();  // Closing the Scanner
            System.out.println("Program execution completed.");
        }
    }
}
```

---

3. Custom Exception (User-defined Exception)

💡 **Problem Statement:**

Create a **custom exception** called `InvalidAgeException`.

- Write a method `validateAge(int age)` that throws `InvalidAgeException` if the age is below 18.
- In `main()`, take user input and call `validateAge()`.

- If an exception occurs, display `"Age must be 18 or above"`.

Expected Behavior:

- If the age is `>=18`, print `"Access granted!"`.
- If age `<18`, throw `InvalidAgeException` and display the message.

```
package com.customexception;

// Defining the custom exception class
public class InvalidAgeException extends Exception {
   // Constructor for custom exception
   public InvalidAgeException(String message) {
       super(message);
   }
}
```

```
package com.customexception;

import java.util.Scanner;

public class CustomException {
   // Method to validate age
   public static void validateAge(int age) throws InvalidAgeException {
       if (age < 18) {
           // Throwing custom exception if age is below 18
           throw new InvalidAgeException("Age must be 18 or above.");
       } else {
           // Printing access granted message if age is valid
           System.out.println("Access granted!");
       }
   }

   public static void main(String[] args) {
       Scanner input = new Scanner(System.in);

       try {
           // Asking the user to enter their age
           System.out.println("Enter your age: ");
```

```
            int age = input.nextInt();

            // Calling the validateAge method
            validateAge(age);

        } catch (InvalidAgeException e) {
            // Handling the custom exception and displaying the message
            System.out.println("Exception caught: " + e.getMessage());
        } catch (Exception e) {
            // Handling other exceptions
            System.out.println("Error: Invalid input.");
        } finally {
            // Closing the Scanner
            input.close();
            System.out.println("Program execution completed.");
        }
    }
}
```

```
package com.customexceptiontest;

import com.customexception.CustomException;
import com.customexception.InvalidAgeException;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CustomExceptionTest {

    @Test
    void testValidAge() {
        // Testing with a valid age (>= 18)
        assertDoesNotThrow(() -> CustomException.validateAge(20), "Age 20 should not
throw an exception");
    }

    @Test
    void testInvalidAge() {
        // Testing with an invalid age (< 18)
        InvalidAgeException exception = assertThrows(InvalidAgeException.class, ()
-> {
            CustomException.validateAge(16);
        });
```

```
        assertEquals("Age must be 18 or above.", exception.getMessage(), "Exception
message should match");
    }

    @Test
    void testBoundaryAge() {
        // Testing with the boundary age (18)
        assertDoesNotThrow(() -> CustomException.validateAge(18), "Age 18 should not
throw an exception");
    }
}
```

---

4. Multiple Catch Blocks

💡 **Problem Statement:**

Create a Java program that performs array operations.

- Accept an integer array and an index number.
- Retrieve and print the value at that index.
- Handle the following exceptions:
    - **`ArrayIndexOutOfBoundsException`** if the index is out of range.
    - **`NullPointerException`** if the array is `null`.

Expected Behavior:

- If valid, print `"Value at index X: Y"`.
- If the index is out of bounds, display `"Invalid index!"`.
- If the array is null, display `"Array is not initialized!"`.

```
package com.multiplecatchblocks;
```

```java
import java.util.Arrays;
import java.util.Scanner;

public class ArrayOperations {
   public static void main(String[] args) {
       Scanner input = new Scanner(System.in);
       // Initializing array as null
       int[] array = null;

       try {
           // Asking user to input array size
           System.out.println("Enter the size of the array: ");
           int size = input.nextInt();

           // Creating the array with the given size
           array = new int[size];

           // Filling the array with user input
           System.out.println("Enter " + size + " elements:");
           for (int i = 0; i < size; i++) {
               array[i] = input.nextInt();
           }

           // Asking for the index to retrieve the value
           System.out.println("Enter the index to retrieve the value: ");
           int index = input.nextInt();

           // Retrieving and printing the value at the specified index
           int value = getValueAtIndex(array, index);
           System.out.println("Value at index " + index + ": " + array[index]);

       } catch (ArrayIndexOutOfBoundsException e) {
           // Handling case when index is out of bounds
           System.out.println("Invalid index!");
       } catch (NullPointerException e) {
           // Handling case when the array is null
           System.out.println("Array is not initialized!");
       } catch (Exception e) {
           // Handling any other unexpected exceptions
           System.out.println("An unexpected error occurred.");
       } finally {
           // Closing the Scanner
           input.close();
           System.out.println("Program execution completed.");
       }
```

```
    }
    // Method to retrieve a value from the array at the specified index
    public static int getValueAtIndex(int[] array, int index) throws
ArrayIndexOutOfBoundsException, NullPointerException {
        return array[index];
    }
}
```

```
package com.multiplecatchblockstest;
import com.multiplecatchblocks.ArrayOperations;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ArrayOperationsTest {

    @Test
    void testValidIndex() {
        int[] array = {1, 2, 3, 4, 5};
        int value = ArrayOperations.getValueAtIndex(array, 2);
        assertEquals(3, value, "Value at index 2 should be 3");
    }

    @Test
    void testArrayIndexOutOfBounds() {
        int[] array = {1, 2, 3, 4, 5};
        assertThrows(ArrayIndexOutOfBoundsException.class, () -> {
            ArrayOperations.getValueAtIndex(array, 10);
        }, "Should throw ArrayIndexOutOfBoundsException for index 10");
    }

    @Test
    void testNullArray() {
        int[] array = null;
        NullPointerException exception = assertThrows(NullPointerException.class, ()
-> {
            ArrayOperations.getValueAtIndex(array, 0);
        }, "should throw NullPointerException.");
    }
}
```

5. try-with-resources (Auto-closing Resources)

💡 **Problem Statement:**

Write a Java program that reads the first line of a file named `"info.txt"` using `BufferedReader`.

- Use **`try-with-resources`** to ensure the file is automatically closed after reading.
- Handle any `IOException` that may occur.

Expected Behavior:

- If the file exists, print its first line.
- If the file does not exist, catch `IOException` and print `"Error reading file"`.

```
package com.trywithresources;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileWithResources {
   public static void main(String[] args) {
       // Using try-with-resources to ensure resources are closed automatically
       try (BufferedReader reader = new BufferedReader(new
FileReader("src/main/java/com/trywithresources/info.txt"))) {
           // Reading the first line of the file
           String firstLine = reader.readLine();

           // Checking if the file is empty
           if (firstLine != null) {
```

```
                System.out.println("First line: " + firstLine);
            } else {
                System.out.println("File is empty.");
            }
        } catch (IOException e) {
            // Handling IOException if the file does not exist or there is an error
reading it
            System.out.println("Either file not found or Error in reading file.");
        }
    }
}
```

```
package com.trywithresources;
import org.junit.jupiter.api.Test;
import java.io.*;
import static org.junit.jupiter.api.Assertions.*;

class ReadFileWithResourcesTest {

    @Test
    void testFileWithContent() throws IOException {
        // Create a temporary file with content
        File tempFile = File.createTempFile("testFile", ".txt");
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(tempFile))) {
            writer.write("Hello, this is a test file.");
        }

        // Read the temporary file
        try (BufferedReader reader = new BufferedReader(new FileReader(tempFile))) {
            String firstLine = reader.readLine();
            assertEquals("Hello, this is a test file.", firstLine, "First line
should match the written content.");
        } finally {
            // Delete the temporary file
            tempFile.delete();
        }
    }

    @Test
    void testEmptyFile() throws IOException {
        // Create an empty temporary file
        File tempFile = File.createTempFile("emptyFile", ".txt");
```

```
        // Read the temporary file
        try (BufferedReader reader = new BufferedReader(new FileReader(tempFile))) {
            String firstLine = reader.readLine();
            assertNull(firstLine, "First line should be null for an empty file.");
        } finally {
            // Delete the temporary file
            tempFile.delete();
        }
    }

    @Test
    void testFileNotFound() {
        // Attempting to read a non-existent file
        Exception exception = assertThrows(IOException.class, () -> {
            try (BufferedReader reader = new BufferedReader(new
FileReader("non_existent_file.txt"))) {
                reader.readLine();
            }
        });

        assertTrue(exception.getMessage().contains("non_existent_file.txt"),
"Exception message should contain the file name.");
    }
}
```

---

6. throw vs. throws (Exception Propagation)

💡 **Problem Statement:**

Create a method `calculateInterest(double amount, double rate, int years)` that:

- Throws `IllegalArgumentException` if `amount` or `rate` is negative.

- Propagates the exception using `throws` and handles it in `main()`.

Expected Behavior:

- If valid, return and print the calculated interest.
- If invalid, catch and display `"Invalid input: Amount and rate must be positive"`.

```
package com.throwsvsthrow;

public class InterestCalculator {

   // Defining a method that throws IllegalArgumentException if the input is
invalid
   public static double calculateInterest(double amount, double rate, int years)
throws IllegalArgumentException {
       // Checking if amount or rate is negative and throwing an exception
       if (amount < 0 || rate < 0) {
           throw new IllegalArgumentException("Amount and rate must be positive.");
       }

       // Calculating the interest
       return amount * rate * years / 100;
   }

   public static void main(String[] args) {
       try {
           // Calling the calculateInterest method with sample values
           double interest = calculateInterest(1000, -5, 2);
           System.out.println("Calculated Interest: " + interest);
       } catch (IllegalArgumentException e) {
           // Handling the propagated exception and printing a user-friendly
message
           System.out.println("Invalid input: Amount and rate must be positive.");
       }
   }
}
```

```
package com.throwsvsthrowtest;
```

```java
import com.throwsvsthrow.InterestCalculator;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class InterestCalculatorTest {

   @Test
   void testValidInterestCalculation() {
       // Verifying the correct calculation of interest
       double interest = InterestCalculator.calculateInterest(1000, 5, 2);  // 1000
* 5 * 2 / 100
       assertEquals(100.0, interest, "Interest should be 100.0 for valid inputs.");
   }

   @Test
   void testNegativeAmountThrowsException() {
       // Verifying that IllegalArgumentException is thrown for negative amount
       assertThrows(IllegalArgumentException.class, () -> {
           InterestCalculator.calculateInterest(-1000, 5, 2);
       }, "IllegalArgumentException should be thrown for negative amount.");
   }

   @Test
   void testNegativeRateThrowsException() {
       // Verifying that IllegalArgumentException is thrown for negative rate
       assertThrows(IllegalArgumentException.class, () -> {
           InterestCalculator.calculateInterest(1000, -5, 2);
       }, "IllegalArgumentException should be thrown for negative rate.");
   }

   @Test
   void testZeroYears() {
       // Verifying that interest is 0 when years is 0
       double interest = InterestCalculator.calculateInterest(1000, 5, 0);
       assertEquals(0.0, interest,  "Interest should be 0 when years is 0.");
   }
}
```

7. finally Block Execution

💡 **Problem Statement:**

Write a program that performs **integer division** and demonstrates the **`finally` block execution**.

- The program should:
  - Take two integers from the user.
  - Perform division.
  - Handle `ArithmeticException` (if dividing by zero).
  - Ensure `"Operation completed"` is always printed using `finally`.

Expected Behavior:

- If valid, print the result.
- If an exception occurs, handle it and still print `"Operation completed"`.

```
package com.finallyblock;
import java.util.InputMismatchException;
import java.util.Scanner;


public class DivisionOperation {
   public static void main(String[] args) {
       Scanner input = new Scanner(System.in);
       try {
           // Taking two integers from the user
           System.out.print("Enter the first integer: ");
           int num1 = input.nextInt();

           System.out.print("Enter the second integer: ");
           int num2 = input.nextInt();

           // Performing division
           int result = divide(num1, num2);
           System.out.println("Result: " + result);

       } catch (ArithmeticException e) {
```

```
            // Handling division by zero exception
            System.out.println("Error: Division by zero is not allowed.");

        } catch (InputMismatchException e) {
            // Handling non-integer input
            System.out.println("Error: Please enter valid integers.");

        } finally {

            input.close();
            // Ensuring this block always executes
            System.out.println("Operation completed.");
        }
    }

    //method to divide
    public static int divide(int num1, int num2) throws ArithmeticException {
        // Performing division
        int result = num1 / num2;
        return result;
    }
}
```

```
package com.finallyblocktest;

import com.finallyblock.*;
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class DivisionOperationTest {

    @Test
    void testValidDivision() {
        // Testing division with valid input
        int result = DivisionOperation.divide(10, 2);
        assertEquals(5, result, "The result should be 5");
    }

    @Test
    void testDivisionByZero() {
        // Testing division by zero, expecting ArithmeticException
        ArithmeticException exception = assertThrows(ArithmeticException.class, ()
-> {
            DivisionOperation.divide(10, 0);
        });
```

```
        assertEquals("/ by zero", exception.getMessage(), "Exception message should
be '/ by zero'");
    }

    @Test
    void testFinallyBlockExecution() {
        // Using a flag to check if finally block executes
        boolean finallyExecuted = false;

        try {
            DivisionOperation.divide(10, 0);
        } catch (ArithmeticException e) {
            // Ignoring exception for this test
        } finally {
            finallyExecuted = true;
        }

        assertTrue(finallyExecuted, "Finally block should always execute");
    }
}
```

---

8. Exception Propagation in Methods

💡 **Problem Statement:**

Create a Java program with three methods:

- `method1()`: Throws an `ArithmeticException` (`10 / 0`).
- `method2()`: Calls `method1()`.
- `main()`: Calls `method2()` and handles the exception.

Expected Behavior:

- The exception propagates from `method1()` → `method2()` → `main()`.
- Catch and handle it in `main()`, printing `"Handled exception in main"`.

```java
package com.exceptionpropogationinmethod;

public class ExceptionPropagation {

    // method1() throws an ArithmeticException (10 / 0)
    static void method1() {
        System.out.println("Inside method1");
        // Throws ArithmeticException
        int result = 10 / 0;
    }

    // method2() calls method1()
    public static void method2() {
        System.out.println("Inside method2");
        // Exception propagates to method2()
        method1();
    }

    // main() calls method2() and handles the exception
    public static void main(String[] args) {
        try {
            System.out.println("Inside main");
            // Exception propagates to main()
            method2();
        } catch (ArithmeticException e) {
            System.out.println("Handled exception in main: " + e);
        }
        System.out.println("Program continues...");
    }
}
```

```java
package com.exceptionpropogationmethod;

import com.exceptionpropogationinmethod.ExceptionPropagation;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ExceptionPropagationTest {

    @Test
    void testMethod2ThrowsArithmeticException() {
        // Verifying that an ArithmeticException is thrown when method2() is called
```

```
        ArithmeticException exception = assertThrows(ArithmeticException.class, ()
-> ExceptionPropagation.method2());

        // Checking that the exception message contains "/ by zero"
        assertTrue(exception.getMessage().contains("/ by zero"), "Exception message
should contain '/ by zero'");
    }
}
```

---

9. Nested try-catch Block

💡 **Problem Statement:**

Write a Java program that:

- Takes an **array** and a **divisor** as input.
- Tries to access an element at an index.
- Tries to divide that element by the divisor.
- Uses **nested try-catch** to handle:
    - `ArrayIndexOutOfBoundsException` if the index is invalid.
    - `ArithmeticException` if the divisor is zero.

Expected Behavior:

- If valid, print the division result.
- If the index is invalid, catch and display `"Invalid array index!"`.
- If division by zero, catch and display `"Cannot divide by zero!"`.

```
package com.nestedtrycatch;

import java.util.Scanner;
```

```java
public class NestedTryCatch {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        Scanner input = new Scanner(System.in);

        try {
            // Taking user input for index and divisor
            System.out.print("Enter the index of the element you want to access: ");
            int index = input.nextInt();

            try {
                System.out.print("Enter the divisor: ");
                int divisor = input.nextInt();

                try {
                    // Performing division
                    int result = divideElement(numbers, index, divisor);
                    System.out.println("Result of division: " + result);
                } catch (ArithmeticException e) {
                    // Handling division by zero
                    System.out.println("Cannot divide by zero!");
                }

            } catch (ArrayIndexOutOfBoundsException e) {
                // Handling invalid array index
                System.out.println("Invalid array index!");
            }

        } catch (Exception e) {
            // Handling other unexpected exceptions
            System.out.println("An unexpected error occurred: " + e);
        } finally {
            input.close();
            System.out.println("Operation completed.");
        }
    }
    // Method to perform division with an element from the array
    public static int divideElement(int[] array, int index, int divisor) {
        if (array == null) {
            throw new NullPointerException("Array is not initialized!");
        }
        int element = array[index];
        return element / divisor;
    }
}
```

```java
package com.nestedtrycatchtest;
import com.nestedtrycatch.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class NestedTryCatchTest {

   @Test
   void testValidDivision() {
       int[] numbers = {10, 20, 30, 40, 50};
       int result = NestedTryCatch.divideElement(numbers, 2, 10);  // 30 / 10
       assertEquals(3, result, "30 divided by 10 should be 3");
   }

   @Test
   void testArrayIndexOutOfBounds() {
       int[] numbers = {10, 20, 30, 40, 50};
       assertThrows(ArrayIndexOutOfBoundsException.class, () -> {
           NestedTryCatch.divideElement(numbers, 10, 2);  // Invalid index
       }, "Should throw ArrayIndexOutOfBoundsException for index 10");
   }

   @Test
   void testDivisionByZero() {
       int[] numbers = {10, 20, 30, 40, 50};
       assertThrows(ArithmeticException.class, () -> {
           NestedTryCatch.divideElement(numbers, 2, 0);  // Division by zero
       }, "Should throw ArithmeticException when dividing by zero");
   }

   @Test
   void testNullArray() {
       int[] numbers = null;
       NullPointerException exception = assertThrows(NullPointerException.class, ()
-> {
           NestedTryCatch.divideElement(numbers, 0, 1);
       });
       assertEquals("Array is not initialized!", exception.getMessage(), "Exception
message should be 'Array is not initialized!'");
   }
}
```

10. Bank Transaction System (Checked + Custom Exception)

💡 **Problem Statement:**

Develop a **Bank Account System** where:

- `withdraw(double amount)` method:
  - Throws `InsufficientBalanceException` if withdrawal amount exceeds balance.
  - Throws `IllegalArgumentException` if the amount is negative.
- Handle exceptions in `main()`.

Expected Behavior:

- If valid, print `"Withdrawal successful, new balance: X"`.
- If balance is insufficient, throw and handle `"Insufficient balance!"`.
- If the amount is negative, throw and handle `"Invalid amount!"`.

```
package com.banktransactionsystem;

// Creating a custom exception for handling insufficient balance cases
class InsufficientBalanceException extends Exception {
   // Passing a custom message to the Exception class
   public InsufficientBalanceException(String message) {
       super(message);
   }
}

// Defining the BankAccount class for managing account balance and transactions
class BankAccount {
   private double balance;

   // Initializing the account with an initial balance
```

```
    public BankAccount(double initialBalance) {
        // Checking if the initial balance is negative
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be
negative!");
        }
        // Setting the initial balance
        this.balance = initialBalance;
    }

    // Defining the withdraw method for processing withdrawals
    public void withdraw(double amount) throws InsufficientBalanceException {
        // Checking if the withdrawal amount is negative
        if (amount < 0) {
            // Throwing an exception for negative amount
            throw new IllegalArgumentException("Invalid amount!");
        }
        // Checking if there are sufficient funds for the withdrawal
        if (amount > balance) {
            // Throwing a custom exception for low balance
            throw new InsufficientBalanceException("Insufficient balance!");
        }
        // Deducting the withdrawal amount from the current balance
        balance -= amount;
        // Printing the updated balance
        System.out.println("Withdrawal successful, new balance: " + balance);
    }

    // Returning the current balance
    public double getBalance() {
        return balance;
    }
}

// Creating the main class to execute the Bank Transaction System
public class BankTransactionSystem {
    public static void main(String[] args) {
        // Creating a BankAccount object with an initial balance of 1000
        BankAccount account = new BankAccount(1000.0);

        try {
            // Printing the current balance
            System.out.println("Current balance: " + account.getBalance());
            // Trying to withdraw 500 (valid)
            account.withdraw(500.0);
```

```
            // Trying to withdraw 600 (causing an insufficient balance exception)
            account.withdraw(600.0);
        } catch (InsufficientBalanceException e) {
            // Handling InsufficientBalanceException and printing the error message
            System.out.println(e.getMessage());
        } catch (IllegalArgumentException e) {
            // Handling IllegalArgumentException for negative amounts and printing
the error message
            System.out.println(e.getMessage());
        } finally {
            // Printing a message indicating that the transaction process is
completed
            System.out.println("Transaction process completed.");
        }
    }
}
```

```
package com.banktransactionsystem;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class BankTransactionSystemTest {

    // Testing the initial balance setup
    @Test
    void testInitialBalance() {
        BankAccount account = new BankAccount(1000.0);
        assertEquals(1000.0, account.getBalance(), "Checking if initial balance is
set correctly");
    }

    // Testing a successful withdrawal
    @Test
    void testWithdrawSuccess() throws InsufficientBalanceException {
        BankAccount account = new BankAccount(1000.0);
        account.withdraw(500.0);  // Withdrawing a valid amount
        assertEquals(500.0, account.getBalance(), "Checking if balance is updated
after successful withdrawal");
    }

    // Testing withdrawal with insufficient balance
```

```
    @Test
    void testWithdrawInsufficientBalance() {
        BankAccount account = new BankAccount(500.0);
        assertThrows(InsufficientBalanceException.class, () -> {
            account.withdraw(600.0);  // Trying to withdraw more than the balance
        }, "Checking if InsufficientBalanceException is thrown when balance is
insufficient");
    }

    // Testing withdrawal with a negative amount
    @Test
    void testWithdrawNegativeAmount() {
        BankAccount account = new BankAccount(1000.0);
        assertThrows(IllegalArgumentException.class, () -> {
            account.withdraw(-100.0);  // Trying to withdraw a negative amount
        }, "Checking if IllegalArgumentException is thrown for negative withdrawal
amount");
    }

    // Testing initial balance with a negative value
    @Test
    void testInitialBalanceNegative() {
        assertThrows(IllegalArgumentException.class, () -> {
            new BankAccount(-1000.0);  // Creating an account with a negative
initial balance
        }, "Checking if IllegalArgumentException is thrown for negative initial
balance");
    }
}
```